



Sécurité des applications

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction et panorama OWASP Top 10

Sécurité des applications

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Pourquoi la sécurité applicative est un enjeu central

Les applications (web, mobiles, API) constituent aujourd'hui la surface d'attaque la plus exposée et la plus dynamique des systèmes d'information. Contrairement à une infrastructure réseau relativement stable, une application évolue en permanence : nouvelles fonctionnalités livrées régulièrement, dépendances tierces mises à jour (ou non), configuration modifiée au fil des déploiements. Chaque changement est une occasion potentielle d'introduire une faiblesse, ce qui explique pourquoi la sécurité applicative ne peut pas être traitée comme un contrôle ponctuel effectué une fois pour toutes, mais comme un processus continu intégré au cycle de développement.

Trois caractéristiques distinguent la sécurité applicative de la sécurité réseau ou système classique. D'abord, l'**exposition directe** à Internet : une application web ou une API est, par construction, accessible depuis n'importe quel point du réseau mondial, sans le filtrage implicite qu'offre un périmètre réseau traditionnel. Ensuite, la **complexité logique** : une application encode une logique métier riche (calculs, autorisations, flux de données) que des couches réseau ou système ne manipulent pas, ce qui multiplie les points où une hypothèse implicite peut être violée par un utilisateur malveillant. Enfin, la **dépendance à du code tiers** : une application moderne s'appuie typiquement sur de nombreuses bibliothèques externes dont la sécurité échappe en partie au contrôle direct de l'équipe qui développe l'application.

Type d'application	Exposition spécifique
Application web	Interface accessible via navigateur, souvent exposée publiquement, cible privilégiée de XSS, CSRF, injection
Application mobile	Code exécuté côté client potentiellement rétro-analysable, stockage local sensible, communication avec une API back-end
API	Généralement sans interface graphique intermédiaire filtrant les usages, consommée par des clients variés (application web, mobile, systèmes tiers), cible directe de scripts automatisés

Ce module approfondit chaque classe de vulnérabilité déjà introduite de façon plus générale au module *Audit organisation et technique* (chapitre 5), avec un objectif de compréhension technique fine du mécanisme sous-jacent à chaque vulnérabilité et de mise en œuvre effective de contre-mesures, plutôt qu'une simple liste de risques à connaître de nom.

2. Rappel du classement OWASP Top 10 (édition de référence)

Le **OWASP Top 10** (*Open Worldwide Application Security Project*) est le référentiel le plus largement utilisé dans l'industrie pour hiérarchiser les risques de sécurité applicative. Il ne s'agit pas d'une liste figée : elle est révisée périodiquement par la communauté OWASP à partir de données de vulnérabilités réellement remontées par des organisations partenaires et de sondages auprès de praticiens de la sécurité, ce qui en fait un instantané représentatif des risques les plus courants et les plus impactants à un moment donné, plutôt qu'un classement purement théorique.

1. Contrôle d'accès défaillant (*broken access control*)
2. Défaillances cryptographiques
3. Injection
4. Conception non sécurisée (*insecure design*)
5. Mauvaise configuration de sécurité
6. Composants vulnérables et obsolètes
7. Défaillances d'identification et d'authentification
8. Défaillances d'intégrité des données et logiciels
9. Journalisation et surveillance insuffisantes
10. Falsification de requête côté serveur (SSRF)

Ce classement évolue périodiquement en fonction des données réelles de vulnérabilités observées ; il reste la référence pédagogique et professionnelle la plus largement adoptée pour structurer un programme de sécurité applicative. Il est complété, pour les architectures orientées API, par un référentiel dédié — l'*OWASP API Security Top 10* — abordé au chapitre 6 de ce module.

Catégorie OWASP	Chapitre de ce module qui l'approfondit
Injection	Chapitre 2
Défaillances d'identification et d'authentification	Chapitre 3
Contrôle d'accès défaillant (dont certaines formes exploitées côté client)	Chapitres 3 et 4
Mauvaise configuration de sécurité, exposition de données	Chapitre 6 (spécifique aux API)

Toutes les catégories du Top 10 ne sont pas traitées avec la même profondeur dans ce module : celles qui exigent une compréhension technique fine d'un mécanisme d'exploitation (injection, authentification, vulnérabilités côté client) font l'objet d'un chapitre dédié ; les autres restent couvertes de façon plus généraliste dans les modules *Audit organisation et technique* et *Security by design*.

3. Principe général : ne jamais faire confiance aux entrées

Le fil conducteur de la quasi-totalité des vulnérabilités applicatives est le même : **une donnée provenant d'une source non fiable (utilisateur, système tiers) est traitée comme si elle était fiable**, sans validation ni neutralisation appropriée avant d'être utilisée dans un contexte sensible (requête base de données, commande système, page HTML, chemin de fichier). Ce principe unificateur permet d'aborder chaque classe de vulnérabilité du module comme une variation d'un même problème fondamental, appliqué à des contextes différents.

Ce schéma se décompose systématiquement en trois éléments : une **source** (le point d'entrée où une donnée non fiable pénètre dans l'application — paramètre d'URL, champ de formulaire, en-tête HTTP, fichier importé, réponse d'un système tiers), un **flux** (le chemin parcouru par cette donnée à travers le code, potentiellement sans aucune transformation), et un **puits** (*sink*, le point où cette donnée est utilisée dans un contexte où elle peut avoir un effet dangereux si elle n'a pas été validée ou neutralisée). Une vulnérabilité existe précisément lorsqu'un chemin non contrôlé relie une source à un puits sensible.

Source non fiable	Puits sensible	Vulnérabilité si non neutralisé
Champ de formulaire	Requête SQL	Injection SQL (chapitre 2)
Paramètre d'URL	Commande système	Injection de commande (chapitre 2)
Commentaire utilisateur	Page HTML rendue	XSS (chapitre 4)
Nom de fichier fourni par l'utilisateur	Appel système de fichiers	Traversée de répertoire (<i>path traversal</i>)
En-tête HTTP <code>Origin</code> non vérifié	Décision d'autorisation CORS	Mauvaise configuration exploitable (chapitre 6)

Ce cadre d'analyse — identifier la source, tracer le flux, repérer le puits — est une compétence transversale directement réutilisable dans les TP du module, ainsi que dans toute activité d'audit de code ultérieure.

4. Validation, encodage, échappement : trois mécanismes distincts à ne pas confondre

La confusion entre ces trois mécanismes est fréquente, y compris chez des développeurs expérimentés, alors qu'ils répondent à des questions différentes et interviennent à des moments différents du traitement d'une donnée.

Validation répond à la question « cette donnée respecte-t-elle le format attendu ? ». Elle vérifie qu'une entrée est conforme à un format attendu (ex. une adresse e-mail respecte une syntaxe donnée, un identifiant numérique ne contient que des chiffres), idéalement par **liste blanche** (n'accepter que ce qui est explicitement autorisé, ex. « uniquement des lettres, chiffres et tirets ») plutôt que par **liste noire** (tenter de bloquer ce qui est connu comme dangereux — toujours incomplète, car il suffit qu'un attaquant trouve une seule variante non couverte par la liste pour la contourner).

Encodage/échappement contextuel répond à la question « comment transformer cette donnée pour qu'elle soit interprétée comme une donnée inerte, et non comme du code, dans le contexte de sortie visé ? ». Un même caractère peut nécessiter un encodage différent selon le contexte de sortie : un caractère `<` doit être transformé en `<` avant insertion dans du HTML, mais suit une règle différente s'il est inséré dans un attribut HTML, dans une chaîne JavaScript ou dans une URL. Appliquer le mauvais encodage pour le contexte visé — une erreur fréquente — neutralise l'entrée de façon incomplète et laisse la vulnérabilité exploitable.

Paramétrage répond à la question « comment séparer structurellement le code de la donnée, pour qu'aucune interprétation ambiguë ne soit possible ? ». C'est le cas, par exemple, des requêtes préparées en base de données (chapitre 2), où la donnée est transmise au moteur d'exécution dans un canal distinct de la structure de la commande elle-même. C'est la défense la plus robuste des trois, car elle élimine la classe de vulnérabilité par construction plutôt que de tenter de neutraliser après coup une donnée déjà mélangée au code.

Mécanisme	Répond à	Exemple	Limite
Validation	La donnée a-t-elle le bon format ?	Contrôle de format d'un identifiant en liste blanche	Ne protège pas une donnée valide mais dangereuse dans son contexte d'usage
Encodage contextuel	Comment neutraliser la donnée pour ce contexte de sortie précis ?	Échappement HTML avant affichage	Doit être appliqué systématiquement, à chaque point de sortie, dans le bon contexte
Paramétrage	Comment séparer structurellement code et donnée ?	Requête préparée SQL	Ne s'applique pas à tous les contextes (ex. pas de paramétrage natif pour l'insertion HTML)

Ces trois mécanismes sont complémentaires plutôt que substituables : une application robuste valide ses entrées en amont, encode systématiquement ses sorties selon le contexte, et privilégie le paramétrage chaque fois qu'un interpréteur (SQL, shell, moteur de gabarit) est sollicité.

5. Défense en profondeur appliquée à la sécurité applicative

Rappel du module *Security by design* : aucune contre-mesure unique n'est infaillible, qu'il s'agisse d'une validation d'entrée, d'un pare-feu applicatif ou d'un contrôle d'accès. Une application robuste combine plusieurs couches de défense indépendantes, de sorte qu'une défaillance isolée d'une couche ne suffise pas, à elle seule, à compromettre l'ensemble du système : c'est le principe de **défense en profondeur** (*defense in depth*).

Appliqué spécifiquement à la sécurité applicative, ce principe se traduit par la combinaison de plusieurs types de contrôles, opérant à des niveaux différents :

Couche	Exemple de contrôle	Objectif
Entrée	Validation stricte, liste blanche	Rejeter les données manifestement invalides avant tout traitement
Traitement	Requêtes paramétrées, API d'exécution sans interprétation shell	Empêcher structurellement l'injection
Sortie	Encodage contextuel systématique	Neutraliser une donnée avant qu'elle n'atteigne un interpréteur (HTML, JavaScript)
Accès	Contrôle d'accès systématique, moindre privilège	Limiter ce qu'un utilisateur ou un composant compromis peut réellement atteindre
Détection	Journalisation, surveillance, alerte	Repérer une exploitation en cours ou déjà survenue

L'intérêt pédagogique de cette grille est de montrer qu'aucune de ces couches, prise isolément, n'est suffisante : une validation d'entrée imparfaite reste rattrapable par un encodage de sortie correct ; un compte de service disposant du moindre privilège limite l'impact même en cas d'injection réussie. C'est cette logique de couches indépendantes et redondantes, plutôt que la recherche d'une solution unique et parfaite, qui structure l'ensemble des contre-mesures présentées dans ce module.

6. Méthodologie de ce module

Chaque chapitre suivant traite une famille de vulnérabilités selon la même structure pédagogique, choisie pour faciliter la comparaison entre familles de vulnérabilités a priori très différentes :

1. **Mécanisme technique** : comment la vulnérabilité fonctionne précisément, au niveau du code ou du protocole concerné — sans cette compréhension technique, une contre-mesure reste une recette appliquée sans discernement, incapable de s'adapter à une variante non prévue.
2. **Exemple d'exploitation** : une illustration concrète et pédagogique du mécanisme, destinée à faire comprendre l'effet réel de la vulnérabilité plutôt qu'à fournir un outil d'attaque directement réutilisable contre un système réel.
3. **Impact** : les conséquences possibles pour l'organisation (confidentialité, intégrité, disponibilité des données ou du système), à mettre en regard des critères DIC déjà vus au module *Introduction à la sécurité*.
4. **Contre-mesures** : les défenses reconnues comme efficaces par la communauté de sécurité, en distinguant si possible une défense structurelle (qui élimine la vulnérabilité par construction) de mesures d'atténuation complémentaires.

Chapitre	Famille de vulnérabilités
2	Injection (SQL, commande, autres interpréteurs)
3	Authentification et gestion de sessions
4	Vulnérabilités côté client (XSS, CSRF, clickjacking)
5	Vulnérabilités mémoire bas niveau et exploitation
6	Sécurisation des API et bonnes pratiques transverses

Les TP associés à chaque chapitre permettent une mise en pratique contrôlée sur des cibles pédagogiques dédiées, conçues spécifiquement pour illustrer un mécanisme sans risque pour un système réel : la compréhension acquise en cours y est mobilisée pour identifier une vulnérabilité, en mesurer l'impact dans un environnement maîtrisé, puis mettre en œuvre et vérifier une contre-mesure appropriée.

À retenir

- L'OWASP Top 10 structure ce module ; chaque chapitre suivant approfondit une ou plusieurs de ses catégories.
- Le principe unificateur des vulnérabilités applicatives est le traitement d'une entrée non fiable comme si elle était fiable.
- Validation par liste blanche, encodage contextuel et paramétrage sont trois mécanismes de défense distincts, souvent combinés.

Chapitre 2 — Vulnérabilités d'injection

Sécurité des applications

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Principe général de l'injection

Une injection se produit lorsqu'une donnée fournie par un utilisateur est concaténée directement dans une commande, une requête ou un interpréteur, sans séparation stricte entre code et donnée, permettant à l'attaquant de modifier la structure ou la sémantique de la commande exécutée. Le point commun de toutes les variantes d'injection — SQL, commande système, LDAP, XPath, NoSQL, gabarit — est ce même défaut structurel : un interpréteur reçoit un texte censé être une simple donnée, mais dans lequel une partie est réinterprétée comme faisant partie de la syntaxe de contrôle du langage cible.

Ce défaut apparaît chaque fois qu'une application construit dynamiquement une commande destinée à un interpréteur (moteur de base de données, shell système, moteur de gabarit, annuaire LDAP...) en assemblant, par simple concaténation de chaînes de caractères, un texte fixe écrit par le développeur et une donnée fournie par l'utilisateur. L'interpréteur cible ne dispose d'aucun moyen de distinguer, dans le texte final qu'il reçoit, ce qui provient du développeur de ce qui provient de l'utilisateur : il applique ses propres règles de syntaxe à l'ensemble du texte, sans distinction d'origine. Un attaquant qui comprend la syntaxe de l'interpréteur visé peut alors construire une entrée qui, une fois insérée dans le texte final, modifie le sens de la commande plutôt que de se contenter d'en être un simple paramètre.

L'injection reste l'une des classes de vulnérabilité les plus anciennes et les mieux documentées du domaine, précisément parce que son mécanisme — mélange non contrôlé de code et de donnée — est simple à comprendre une fois explicité, ce qui en fait un cas d'étude particulièrement pédagogique pour illustrer le principe général vu au chapitre 1 (ne jamais faire confiance aux entrées).

Le `--` commente le reste de la requête (syntaxe SQL) : la vérification du mot de passe est neutralisée, permettant une connexion en tant qu'`admin` sans en connaître le mot de passe. Ce cas illustre l'essence même de l'injection : le guillemet simple contenu dans l'entrée `login` referme prématurément la chaîne de caractères attendue par le développeur, et le texte qui suit (`--`) est alors interprété par le moteur SQL comme faisant partie de la structure de la requête, et non plus comme une simple valeur.

Injection UNION-based

Une injection peut aussi être utilisée pour extraire des données d'autres tables via l'opérateur `UNION SELECT`, si le nombre et le type de colonnes de la requête d'origine sont devinés ou déduits par tâtonnement (technique pratiquée en TP1). Le principe consiste à faire en sorte que la requête modifiée renvoie, en plus (ou à la place) du résultat attendu, les lignes d'une seconde requête choisie par l'attaquant et combinée via `UNION SELECT`, à condition que le nombre de colonnes et leurs types soient compatibles entre les deux requêtes. Cette technique est particulièrement redoutée car elle permet, dans une application qui affiche directement le résultat d'une requête, d'exfiltrer des données de tables normalement inaccessibles via l'interface prévue.

Injection en aveugle (blind SQL injection)

Lorsque l'application ne retourne pas directement le résultat de la requête mais seulement un comportement différent (page d'erreur, temps de réponse), un attaquant peut néanmoins extraire des données bit par bit en observant ces différences. On distingue deux sous-variantes : l'**injection booléenne**, où l'attaquant formule des conditions vraies ou fausses et observe une différence de comportement de la page (contenu affiché, présence ou absence d'un élément) selon que la condition est vérifiée ou non ; et l'**injection temporelle**, où l'attaquant provoque un délai de réponse mesurable (par exemple via une fonction de pause du moteur de base de données) uniquement lorsqu'une condition est vraie, permettant de déduire une information bit par bit à partir du seul temps de réponse observé, même en l'absence de toute différence visible dans le contenu de la réponse.

Variante	Signal observé par l'attaquant	Contexte typique
UNION-based	Résultat de requête directement affiché	Page listant des résultats de recherche
Booléenne (blind)	Différence de contenu ou de comportement de la page	Formulaire de connexion, filtre de recherche
Temporelle (blind)	Différence de temps de réponse	Application ne renvoyant aucune différence visible

Contre-mesure de référence : requêtes préparées (paramétrées)

3. Injection de commande système

```
char commande[100];
snprintf(commande, sizeof(commande), "ping -c 1 %s", entree_utilisateur);
system(commande);
```

Si `entree_utilisateur` vaut `8.8.8.8; rm -rf /donnees`, la commande système exécutée exécute en réalité deux commandes distinctes, la seconde étant totalement contrôlée par l'attaquant. Le mécanisme sous-jacent est le même que pour l'injection SQL : la fonction `system()` transmet le texte reçu à un interpréteur de commande shell, qui applique ses propres règles de syntaxe (point-virgule, esperluette, tube `|`, apostrophe inversée) à l'ensemble du texte, sans distinguer ce qui provient du développeur de ce qui provient de l'utilisateur.

Cette classe de vulnérabilité se retrouve, sous des formes équivalentes, dans la plupart des langages : `system()` et `popen()` en C, `os.system()` ou `subprocess` appelé avec `shell=True` en Python, `Runtime.exec()` mal utilisé en Java, `exec()` en PHP ou en Node.js. Le point commun est le passage par un interpréteur shell intermédiaire, qui interprète les métacaractères de contrôle plutôt que de traiter l'ensemble du texte comme un argument unique et inerte.

Contre-mesure : éviter autant que possible d'appeler un interpréteur de commande shell avec des données utilisateur ; si indispensable, utiliser des API d'exécution de processus qui séparent explicitement la commande de ses arguments (par exemple en passant les arguments sous forme de liste plutôt que de chaîne unique, sans passer par un shell interprétant les métacaractères), et valider strictement le format attendu (liste blanche, ex. une adresse IP doit correspondre à un motif numérique précis). Comme pour l'injection SQL, la séparation structurelle entre commande et arguments élimine la vulnérabilité par construction, alors qu'un filtrage de caractères dangereux reste toujours susceptible d'oublier un cas.

4. Autres formes d'injection

Le principe d'injection ne se limite pas au SQL et au shell système : tout interpréteur recevant un texte partiellement construit à partir de données non fiables est potentiellement vulnérable, avec des conséquences qui varient selon le rôle de l'interpréteur ciblé dans l'application.

Type	Contexte	Exemple
LDAP injection	annuaires d'authentification	manipulation d'un filtre de recherche LDAP pour altérer la logique d'authentification ou de recherche
XPath injection	requêtes sur documents XML	équivalent de l'injection SQL pour XPath, permettant de contourner une condition de sélection de nœuds
NoSQL injection	bases de données non relationnelles	injection d'opérateurs de requête (ex. un objet au lieu d'une chaîne attendue) dans un document JSON mal validé, modifiant la logique de la requête
Injection de gabarit côté serveur (SSTI)	moteurs de templates web	exécution de code via une syntaxe de template non neutralisée, lorsqu'une donnée utilisateur est interprétée comme faisant partie du gabarit plutôt que comme une simple valeur à afficher
Injection de code (command/eval injection)	fonctions d'évaluation dynamique de code	passage direct d'une entrée utilisateur à une fonction <code>eval()</code> ou équivalente, qui interprète la chaîne reçue comme du code exécutable dans le langage de l'application elle-même

Ces variantes partagent une propriété importante : leur découverte nécessite de comprendre la syntaxe précise de l'interpréteur ciblé (LDAP, XPath, un langage de gabarit spécifique), ce qui explique pourquoi elles sont souvent moins bien couvertes par les outils de détection automatisée génériques que l'injection SQL, plus universellement connue et outillée.

5. Principe de défense commun

Quelle que soit la variante, la défense structurelle privilégiée est la **séparation stricte entre code et donnée** (requêtes paramétrées, API d'exécution sans interprétation shell, désactivation des fonctionnalités dynamiques dangereuses des moteurs de template), complétée par une validation stricte des entrées en liste blanche et le principe du **moindre privilège** pour le compte utilisé par l'application (un compte de base de données en lecture seule limite l'impact même en cas d'injection réussie).

Ce principe de moindre privilège mérite d'être développé : même lorsqu'une injection SQL parvient à contourner toutes les défenses en amont, ses conséquences restent bornées par les droits effectifs accordés au compte de base de données utilisé par l'application. Un compte disposant uniquement d'un accès en lecture sur les tables strictement nécessaires ne permettra jamais, même en cas d'injection réussie, de modifier ou de supprimer des données, ni d'accéder à des tables sans rapport avec la fonctionnalité concernée. Ce raisonnement illustre concrètement la logique de défense en profondeur introduite au chapitre 1 : chaque couche de défense (validation, paramétrage, moindre privilège, journalisation) réduit l'impact d'une éventuelle défaillance des autres couches, plutôt que de reposer sur une seule mesure supposée infaillible.

À retenir

- L'injection résulte du mélange entre code et donnée non fiable dans un même flux interprété.
- Les requêtes préparées constituent la défense de référence contre l'injection SQL, structurellement plus robuste qu'un simple échappement.
- Le principe de moindre privilège du compte applicatif limite l'impact même en cas de contournement des autres défenses.

Chapitre 3 — Authentification et gestion de sessions

Sécurité des applications

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

répond à une question différente (« cet utilisateur a-t-il le droit d'effectuer cette action ? ») et qui intervient nécessairement après une authentification réussie. Une confusion fréquente en pratique — traiter une authentification réussie comme suffisante pour autoriser n'importe quelle action — est à l'origine d'une part importante des défaillances de contrôle d'accès rencontrées en audit.

Stockage des mots de passe

Rappel des modules *Cryptanalyse* et *Cryptographie* : un mot de passe ne doit jamais être stocké en clair, ni haché avec une fonction générique rapide seule (SHA-256 brut). Une fonction de hachage générique est conçue pour être rapide à calculer, une propriété recherchée pour l'intégrité de fichiers mais directement contre-productive pour le stockage de mots de passe : elle permet à un attaquant disposant d'une base de hachages dérobée de tester des milliards de mots de passe candidats par seconde sur du matériel courant. Il faut utiliser une fonction dédiée et volontairement lente, salée (**bcrypt**, **scrypt**, **Argon2**), dont le facteur de coût (nombre d'itérations, mémoire requise) peut être ajusté pour rester coûteux à mesure que la puissance de calcul disponible augmente. Le **sel** (une valeur aléatoire unique par utilisateur, combinée au mot de passe avant hachage) empêche par ailleurs qu'une même valeur de mot de passe produise le même hachage pour deux utilisateurs différents, ce qui neutralise les attaques par table précalculée (*rainbow tables*).

Politique de mots de passe

Les recommandations actuelles (NIST SP 800-63B notamment) privilégient une **longueur minimale suffisante** (12 caractères ou plus) plutôt que des règles de composition complexes peu efficaces en pratique : les utilisateurs contournent des règles trop contraignantes (au moins une majuscule, un chiffre, un symbole) par des motifs prévisibles (`Motdepasse1!`), qui satisfont la règle sans réellement augmenter la résistance du mot de passe à une attaque par dictionnaire. Les recommandations actuelles privilégient également la **vérification contre des listes de mots de passe déjà compromis** lors de la création d'un compte (rejeter un mot de passe connu comme ayant déjà fuité ailleurs, même s'il respecte par ailleurs les critères de format), et la **suppression de l'expiration périodique forcée** sans raison particulière : une mesure historiquement répandue mais aujourd'hui considérée contre-productive, car elle pousse les utilisateurs à des variations prévisibles du même mot de passe d'un renouvellement à l'autre (`Motdepasse1`, puis `Motdepasse2`), sans gain réel de sécurité.

Authentification multi-facteurs (MFA)

Combine au moins deux catégories parmi : ce que l'utilisateur **sait** (mot de passe, code PIN), ce qu'il **possède** (application d'authentification générant des codes temporaires, clé de sécurité physique), ce qu'il **est** (biométrie). L'intérêt de combiner plusieurs catégories plutôt qu'un seul facteur renforcé est que les catégories reposent sur des mécanismes de compromission différents : un mot de passe divulgué par phishing ne suffit plus à un attaquant qui ne possède pas également le **second facteur**. Le MFA réduit ainsi fortement l'impact d'un mot de passe compromis (par phishing, fuite de données, réutilisation d'un même mot de passe sur plusieurs services), sans pour autant éliminer tout risque : certaines

2. Défaillances d'authentification fréquentes

Au-delà du choix du mécanisme d'authentification lui-même, de nombreuses défaillances proviennent de la façon dont ce mécanisme est mis en œuvre autour du processus de connexion :

Défaillance	Risque
Absence de limitation du nombre de tentatives	attaque par force brute ou par dictionnaire praticable, un attaquant pouvant tester un grand nombre de mots de passe sans être bloqué
Messages d'erreur distinguant « utilisateur inconnu » de « mot de passe incorrect »	permet à un attaquant d'énumérer les comptes existants, information utile pour cibler ensuite une attaque par dictionnaire ou du phishing
Absence de MFA sur les comptes à privilèges	un mot de passe compromis suffit à compromettre un compte critique (administration, accès à des données sensibles)
Réinitialisation de mot de passe mal sécurisée	jeton de réinitialisation prévisible, réutilisable ou sans expiration, permettant une prise de contrôle de compte sans connaître le mot de passe d'origine
Credential stuffing non contrôlé	réutilisation automatisée de couples identifiant/mot de passe issus de fuites de données antérieures sur d'autres services, exploitant la réutilisation fréquente d'un même mot de passe par un utilisateur

Ces défaillances partagent un point commun : elles concernent rarement l'algorithme cryptographique lui-même (correctement traité au module *Cryptographie*), mais plutôt la logique applicative qui l'entoure — ce qui explique pourquoi elles restent fréquentes même dans des applications utilisant par ailleurs des mécanismes cryptographiques solides. Une limitation de tentatives (*rate limiting*, approfondie au chapitre 6) combinée à une journalisation des échecs d'authentification répétés permet de détecter et de freiner la plupart de ces attaques automatisées.

3. Gestion de sessions

Après authentification, un **identifiant de session** (généralement transmis via un cookie) permet à l'application de reconnaître l'utilisateur sur les requêtes suivantes sans redemander ses identifiants à chaque fois. Le principe général reste le même que pour un mot de passe : cet identifiant devient, le temps de sa validité, l'équivalent fonctionnel des identifiants de connexion — sa compromission équivaut à une usurpation complète de l'identité de l'utilisateur pour la durée de la session, sans même que l'attaquant ait besoin de connaître le mot de passe.

Exigences de sécurité d'un identifiant de session

- **Imprévisibilité** : généré par un générateur aléatoire cryptographiquement sûr, suffisamment long pour résister à une attaque par force brute cherchant à deviner un identifiant valide parmi l'espace des valeurs possibles.
- **Régénération après authentification** : un identifiant de session attribué avant authentification ne doit jamais rester valide après (prévention de la **fixation de session**, où un attaquant impose à sa victime un identifiant de session qu'il connaît à l'avance — par exemple via un lien contenant cet identifiant — puis attend que la victime s'authentifie pour réutiliser ce même identifiant, désormais associé à une session authentifiée).
- **Expiration** : durée de vie limitée, avec expiration après une période d'inactivité et invalidation explicite côté serveur à la déconnexion (une simple suppression du cookie côté client, sans invalidation côté serveur, laisse l'identifiant de session encore valide s'il a été intercepté par ailleurs).
- **Attributs de cookie sécurisés** : `Secure` (transmission uniquement en HTTPS, empêchant l'interception en clair sur un réseau non chiffré), `HttpOnly` (inaccessible au JavaScript côté client, limitant l'impact d'un XSS — chapitre 4, puisqu'un script injecté ne peut alors pas lire directement la valeur du cookie), `SameSite` (limite l'envoi automatique du cookie lors de requêtes intersites, contre-mesure au CSRF — chapitre 4).

Attribut de cookie	Protège contre
<code>Secure</code>	Interception réseau en clair
<code>HttpOnly</code>	Lecture du cookie par un script injecté (XSS)
<code>SameSite</code>	Envoi automatique du cookie lors d'une requête intersite forgée (CSRF)

Ces trois attributs sont complémentaires : chacun neutralise un vecteur d'attaque différent, et leur combinaison illustre une nouvelle fois la logique de défense en profondeur — un identifiant de session bien généré mais dépourvu de ces attributs reste exposé aux vulnérabilités traitées au chapitre suivant.

4. Authentification fédérée et jetons (aperçu)

De nombreuses applications modernes délèguent l'authentification à un fournisseur d'identité tiers (**OAuth 2.0**, **OpenID Connect**) plutôt que de gérer elles-mêmes des mots de passe, ce qui présente un intérêt de sécurité réel : l'application n'a plus à stocker ni à protéger elle-même des identifiants, cette responsabilité étant concentrée chez un fournisseur spécialisé dont c'est le métier principal. Ces protocoles introduisent néanmoins leurs propres risques (mauvaise validation d'un jeton, absence de vérification de la signature d'un JWT, confusion entre autorisation et authentification — OAuth 2.0 est à l'origine un protocole d'autorisation, détourné en pratique pour de l'authentification via OpenID Connect qui le complète explicitement à cet effet) qui nécessitent une compréhension précise du protocole utilisé plutôt qu'une implémentation ad hoc.

JWT (JSON Web Token) : points de vigilance

Un JWT se compose de trois parties encodées et séparées par un point (en-tête, charge utile, signature). Sa vérification correcte est une source fréquente d'erreurs :

- Toujours vérifier la **signature** du jeton reçu, avec l'algorithme attendu explicitement fixé côté serveur (ne jamais faire confiance à l'algorithme annoncé dans l'en-tête du jeton lui-même — une classe de vulnérabilité réelle où un attaquant modifie l'algorithme annoncé en `none` ou substitue une clé publique en clé symétrique pour forger une signature valide sans connaître la clé secrète du serveur).
- Vérifier systématiquement l'expiration (`exp`) et l'émetteur attendu (`iss`), pour éviter qu'un jeton expiré ou émis par un fournisseur d'identité différent de celui attendu ne soit accepté à tort.
- Ne jamais stocker d'informations sensibles non chiffrées dans le contenu d'un JWT, qui n'est qu'**encodé** (Base64), pas chiffré, et donc lisible en clair par quiconque intercepte le jeton — une erreur fréquente consiste à confondre encodage (réversible sans clé) et chiffrement (nécessitant une clé pour être inversé).

À retenir

- Une politique de mot de passe moderne privilégie la longueur et la vérification contre des mots de passe déjà compromis plutôt que des règles de composition contraignantes et peu efficaces.
- La gestion de session exige régénération après authentification, expiration maîtrisée, et attributs de cookie sécurisés (Secure, HttpOnly, SameSite).
- L'authentification fédérée (OAuth/OIDC/JWT) déplace le risque vers une bonne compréhension du protocole et une vérification rigoureuse des jetons, plutôt que de l'éliminer.

Chapitre 4 — Vulnérabilités côté client : XSS, CSRF, clickjacking

Sécurité des applications

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

du navigateur, qui interprète différemment un même texte selon qu'il est traité comme une donnée à afficher ou comme du balisage à exécuter.

Trois formes principales

Type	Mécanisme
XSS réfléchi	la charge malveillante est incluse dans la requête (ex. paramètre d'URL) et renvoyée immédiatement dans la réponse, sans être stockée ; nécessite de piéger la victime pour qu'elle clique un lien spécialement construit
XSS stocké (persistant)	la charge est enregistrée côté serveur (ex. commentaire, champ de profil) et exécutée pour tout utilisateur consultant ultérieurement le contenu affecté ; impact généralement plus large, car aucune action spécifique de la victime au-delà de la simple consultation de la page n'est nécessaire
XSS DOM-based	la vulnérabilité provient d'un traitement non sûr côté client (JavaScript manipulant directement le DOM à partir de données non fiables, par exemple un fragment d'URL lu et réinjecté dans la page), sans que le serveur soit nécessairement impliqué dans la faille ni même conscient de la donnée concernée

Ces trois formes se distinguent essentiellement par l'endroit où la donnée non fiable transite avant d'atteindre le puits sensible (immédiatement dans la réponse, via un stockage persistant, ou entièrement côté client), ce qui a une conséquence directe sur la façon de les détecter : une analyse du code serveur suffit à repérer les deux premières formes, alors que la troisième nécessite d'examiner le code JavaScript exécuté côté navigateur.

Exemple

```
<!-- Champ de commentaire affiché sans encodage -->
<p>Commentaire : <?php echo $commentaire_utilisateur; ?></p>
```

Si `$commentaire_utilisateur` contient une balise `<script>` insérée sans échappement, le code JavaScript qu'elle contient s'exécute dans le navigateur de tout visiteur affichant ce commentaire, avec accès potentiel au cookie de session de la victime (d'où l'importance de l'attribut `HttpOnly` vu au chapitre 3, qui limite précisément ce vecteur en rendant le cookie inaccessible à un script, même exécuté avec succès). Le défaut ici est l'absence totale d'encodage contextuel entre la donnée fournie par l'utilisateur et son insertion dans le flux HTML renvoyé au navigateur : le moteur de rendu ne dispose d'aucun moyen de savoir que ce fragment de texte devait rester une donnée inerte plutôt que d'être traité comme du balisage actif.

Contre-mesures

Un attaquant piège la victime (déjà authentifiée sur un site cible) pour qu'elle exécute, à son insu, une requête vers ce site (ex. via une page piégée provoquant une soumission de formulaire automatique) : le navigateur joint automatiquement les cookies de session valides à toute requête vers le domaine concerné, quel que soit le site à l'origine de la requête, faisant apparaître la requête comme légitime aux yeux du serveur. C'est précisément ce comportement par défaut du navigateur — joindre automatiquement les cookies pertinents à chaque requête, indépendamment de son origine — qui rend possible cette attaque : le serveur reçoit une requête indiscernable, sur la seule base des cookies, d'une requête réellement initiée par l'utilisateur depuis le site légitime.

Contrairement à XSS, le CSRF n'exploite aucune faille de rendu ou d'exécution de code dans l'application cible : il exploite la confiance automatique et implicite que le navigateur accorde, par défaut, à toute requête intersite envoyée vers un domaine pour lequel des cookies existent déjà.

Exemple

```
<!-- Page piégée hébergée par l'attaquant -->
<form action="https://banque.example/virement" method="POST">
  <input type="hidden" name="destinataire" value="compte-attaquant">
  <input type="hidden" name="montant" value="5000">
</form>
<script>document.forms[0].submit();</script>
```

Si la victime authentifiée sur `banque.example` visite cette page piégée, son navigateur soumet automatiquement le formulaire avec ses cookies de session valides — la requête reçue par `banque.example` est, du point de vue du serveur, syntaxiquement identique à celle qu'aurait envoyée l'utilisateur légitime en remplissant volontairement le formulaire de virement, ce qui empêche une simple vérification de format de la requête de distinguer les deux cas.

Contre-mesures

- **Jeton anti-CSRF** : valeur unique et imprévisible générée côté serveur, incluse dans chaque formulaire sensible et vérifiée à la soumission — un attaquant externe ne peut pas connaître ce jeton à l'avance (il n'a accès ni à la page légitime contenant le jeton, ni au moyen de le déduire), et une requête forgée sans ce jeton correct est rejetée par le serveur.
- **Attribut de cookie `SameSite`** (rappel chapitre 3) : empêche le navigateur d'envoyer automatiquement le cookie de session lors d'une requête intersite dans de nombreux cas, neutralisant le mécanisme même sur lequel repose l'attaque, sans nécessiter de modification du formulaire

3. Clickjacking

Un attaquant superpose (via une iframe transparente) une page légitime sous une interface trompeuse, incitant la victime à cliquer sur un élément de la page légitime sans le savoir (ex. un bouton « activer le micro » ou « confirmer un paiement » masqué sous un bouton apparemment inoffensif). Le principe repose sur une manipulation purement visuelle : la page légitime, chargée normalement dans une iframe rendue invisible ou transparente par une feuille de style contrôlée par l'attaquant, reçoit réellement le clic de la victime, alors que celle-ci pense interagir avec le contenu visible de la page piégeante superposée. Aucune donnée n'est falsifiée ni interceptée : c'est l'action même de l'utilisateur, légitime en apparence pour le serveur qui la reçoit, qui est détournée.

Contre-mesure : en-tête HTTP `X-Frame-Options` ou directive CSP `frame-ancestors`, empêchant qu'une page sensible soit chargée dans une iframe par un site non autorisé — le navigateur refuse alors purement et simplement d'afficher la page dans l'iframe piégée, rendant l'attaque impossible sans qu'aucune modification de la logique applicative ne soit nécessaire.

4. Injection côté client : DOM et frameworks modernes

Les applications à page unique (SPA, *single-page applications*) manipulant intensivement le DOM côté client introduisent des variantes de ces vulnérabilités, en particulier le XSS DOM-based, qui déplace une partie de la surface d'attaque du code serveur vers le code JavaScript exécuté dans le navigateur. Une source de donnée non fiable, côté client, peut provenir de l'URL elle-même (fragment, paramètre de requête lu directement par du JavaScript), du contenu d'un message reçu via une API de communication entre fenêtres, ou d'une réponse d'API insuffisamment neutralisée avant d'être insérée dans le DOM.

Cette évolution architecturale nécessite une vigilance spécifique sur toute fonction manipulant directement le HTML à partir de données non fiables (fonctions explicitement signalées comme dangereuses par les frameworks modernes — leur nommage explicite vise précisément à attirer l'attention du développeur sur le risque encouru — à éviter sauf nécessité strictement justifiée et accompagnée d'un encodage manuel rigoureux). Le principe de défense reste identique à celui du XSS traditionnel — encodage contextuel systématique avant toute insertion dans le DOM — mais son application se déplace du code serveur vers le code client, ce qui implique d'étendre la vigilance et, le cas échéant, les outils d'analyse statique à l'ensemble de la base de code JavaScript.

À retenir

- XSS exploite une insertion non sécurisée de données dans une page ; CSRF exploite la confiance automatique du navigateur envers des cookies de session lors de requêtes intersites — deux mécanismes distincts nécessitant des contre-mesures distinctes.
- L'encodage contextuel systématique et une CSP bien configurée sont les défenses de référence contre XSS ; le jeton anti-CSRF et l'attribut `SameSite` sont les défenses de référence contre CSRF.
- Le clickjacking se prévient simplement par les en-têtes HTTP appropriés (`X-Frame-Options`, `frame-ancestors`), souvent négligés en pratique.

Chapitre 5 — Vulnérabilités mémoire bas niveau et exploitation

Sécurité des applications

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Retour sur les fondamentaux (lien avec le module Algorithmique et C)

Ce chapitre approfondit, sous l'angle de l'exploitation, les vulnérabilités mémoire déjà identifiées comme risques au module *Algorithmique et Programmation en C* (chapitres 5 et 6) : débordement de tampon, use-after-free, format string. Ces vulnérabilités partagent une caractéristique qui les distingue nettement des vulnérabilités applicatives vues aux chapitres précédents (injection, XSS) : elles résultent d'une **gestion incorrecte de la mémoire** par le programme lui-même, dans des langages (C, C++ notamment) qui laissent au développeur l'entière responsabilité de cette gestion, sans filet de sécurité automatique (contrairement à des langages gérant la mémoire de façon automatisée, où une classe entière de ces défauts est structurellement écartée).

Le fil conducteur reste néanmoins le même qu'au chapitre 1 : une donnée non fiable (une entrée dont la taille ou le contenu n'est pas maîtrisé par le programme) atteint un point sensible (une opération mémoire non bornée) sans validation préalable adéquate. Ici, le « puits sensible » n'est plus un interpréteur de commande mais la mémoire du processus lui-même, ce qui change radicalement la nature de l'impact possible : de la simple altération d'une donnée applicative à la prise de contrôle complète du flot d'exécution du programme.

2. Anatomie d'un débordement de tampon sur la pile (stack overflow)

La pile d'exécution contient, pour chaque appel de fonction, les variables locales et l'**adresse de retour** (l'endroit où l'exécution doit reprendre après la fin de la fonction). Ces éléments sont organisés de façon contiguë en mémoire, dans un ordre déterminé par le compilateur et l'architecture cible ; c'est précisément cette contiguïté qui rend possible l'écrasement d'une zone mémoire par le débordement d'une zone adjacente. Un débordement de tampon local mal borné peut écraser cette adresse de retour :

```
void fonction_vulnerable(char *entree) {  
    char buffer[64];  
    strcpy(buffer, entree); // aucune vérification de taille  
}
```

`strcpy()` copie des octets depuis `entree` vers `buffer` jusqu'à rencontrer un caractère de fin de chaîne, sans jamais vérifier que le nombre d'octets copiés reste inférieur à la taille réelle de `buffer` (ici 64 octets). Si `entree` dépasse cette taille, les octets excédentaires écrasent la mémoire adjacente sur la pile, potentiellement jusqu'à l'adresse de retour. Le principe de l'exploitation consiste, pour un attaquant maîtrisant précisément le contenu de ce dépassement, à rediriger cette adresse de retour vers un emplacement de son choix : historiquement, du code injecté dans le buffer débordant lui-même (technique dite du « shellcode »), aujourd'hui rendue beaucoup plus difficile par les protections systémiques décrites ci-dessous. Ce détournement du flot d'exécution — faire reprendre le programme ailleurs qu'à l'endroit légitime prévu par le code source — est ce qui distingue une vulnérabilité mémoire exploitable d'un simple plantage sans conséquence au-delà d'un déni de service.

3. Contre-mesures systémiques (défense en profondeur au niveau du système)

Les systèmes d'exploitation et compilateurs modernes ont introduit plusieurs mécanismes rendant l'exploitation d'un débordement de tampon bien plus difficile qu'à l'époque des premières attaques historiques (fin des années 1990), sans pour autant l'éliminer complètement — d'où l'intérêt pédagogique de bien distinguer ce que chaque protection empêche réellement de ce qu'elle ne fait que compliquer.

Contre-mesure	Principe
Stack canary	valeur secrète placée entre les variables locales et l'adresse de retour, vérifiée avant le retour de fonction ; toute modification (par débordement) est détectée et provoque l'arrêt immédiat du programme, avant que l'adresse de retour corrompue ne soit effectivement utilisée
ASLR (Address Space Layout Randomization)	randomise l'emplacement en mémoire du tas, de la pile et des bibliothèques à chaque exécution, rendant plus difficile de prédire à l'avance l'adresse exacte vers laquelle rediriger l'exécution, même si l'adresse de retour a bien été écrasée
DEP/NX (Data Execution Prevention / No-eXecute)	marque les zones mémoire de données (dont la pile) comme non exécutables au niveau matériel, empêchant l'exécution directe d'un shellcode injecté dans un buffer, même si le flot d'exécution est effectivement redirigé vers cette zone
RELRO, PIE	durcissements complémentaires : RELRO rend certaines tables de symboles en lecture seule après le chargement du programme, PIE (<i>Position Independent Executable</i>) permet de randomiser également l'emplacement du code exécutable lui-même, et non plus seulement celui des données

Chacune de ces protections cible une étape distincte d'une exploitation réussie (détecter la corruption, empêcher de prédire une adresse, empêcher d'exécuter des données), ce qui illustre une nouvelle fois le principe de défense en profondeur : contourner une seule de ces protections ne suffit généralement plus à obtenir une exploitation fiable, il faut les contourner conjointement. Ces protections ont ainsi donné naissance à des techniques d'exploitation plus avancées, telles que le **retour orienté programmation** (*return-oriented programming*, ROP), qui réutilise des fragments de code déjà présents et déjà marqués exécutables dans le programme ou ses bibliothèques, plutôt que d'injecter du code nouveau, contournant ainsi la protection DEP/NX sans avoir besoin d'exécuter de donnée. Ces techniques avancées font l'objet d'un approfondissement possible au-delà de ce cours introductif.

4. Autres vulnérabilités mémoire exploitables

Le débordement de tampon sur la pile n'est que la forme la plus enseignée, historiquement, de cette famille de vulnérabilités. D'autres schémas, tout aussi présents dans les logiciels réels, exploitent des erreurs de gestion mémoire différentes :

- **Débordement de tampon sur le tas (heap overflow)** : cible des structures de gestion du tas (métadonnées utilisées par l'allocateur mémoire pour suivre les blocs libres et occupés) ou des données adjacentes à un tampon alloué dynamiquement. L'exploitation est généralement plus complexe que sur la pile, car la disposition du tas dépend fortement de l'historique des allocations et désallocations précédentes, mais reste toujours d'actualité (rappel module C, chapitre 6).
- **Use-after-free** : exploitation d'un pointeur pendant (*dangling pointer*) vers une zone mémoire déjà libérée par le programme, mais dont le pointeur n'a pas été mis à jour ni invalidé. Si cette zone est ensuite réallouée à d'autres données par le programme, l'utilisation du pointeur pendant permet potentiellement à un attaquant de lire ou de manipuler des structures de données internes du programme qui n'ont, en apparence, aucun rapport avec le pointeur d'origine.
- **Vulnérabilité de chaîne de format (format string)** : passage d'une entrée utilisateur directement comme chaîne de format à une fonction comme `printf(entree_utilisateur)` (au lieu de la forme correcte `printf("%s", entree_utilisateur)`, qui traite l'entrée comme une simple donnée à afficher). Dans la forme incorrecte, si `entree_utilisateur` contient des spécificateurs de format (`%x`, `%n`), la fonction les interprète comme des instructions de lecture (`%x`, révélant le contenu de la pile) voire d'écriture en mémoire (`%n`, qui écrit le nombre de caractères déjà affichés à une adresse fournie par l'appelant).
- **Débordement d'entier (integer overflow)** : un calcul de taille (par exemple une multiplication du nombre d'éléments par la taille d'un élément, avant un appel d'allocation mémoire) qui déborde silencieusement la capacité du type entier utilisé peut produire une valeur bien plus petite que le résultat mathématiquement attendu, conduisant à une allocation mémoire plus petite que prévu, et provoquant ensuite un débordement classique lors du remplissage de cette zone sous-dimensionnée par le reste du programme.

Ces quatre formes illustrent que la vulnérabilité mémoire ne se limite pas à « écrire trop loin dans un tableau » : elle recouvre toute situation où l'état interne de la gestion mémoire du programme (allocations, libérations, tailles calculées) peut être manipulé, directement ou indirectement, par une donnée non fiable.

5. Démarche d'analyse d'une vulnérabilité mémoire (aperçu, développé en TP3)

1. Identifier le point d'entrée de données non fiables et la fonction dangereuse concernée (fonction non bornée, calcul de taille, gestion de pointeur).
2. Déterminer la structure mémoire environnante (variables locales, ordre en mémoire, présence de protections telles que canary, ASLR, DEP/NX), afin de comprendre ce qui est réellement atteignable par un débordement à cet endroit précis.
3. Construire une entrée de test provoquant un comportement anormal observable (plantage contrôlé), première étape indispensable pour confirmer l'existence réelle de la vulnérabilité avant toute analyse plus poussée.
4. Affiner progressivement pour comprendre précisément l'impact (simple déni de service par plantage, ou contrôle effectif du flot d'exécution), en gardant à l'esprit que cette démarche, en TP, se déroule exclusivement sur des cibles pédagogiques dédiées, conçues pour l'apprentissage et sans lien avec un système en production.

Cette démarche méthodique — identifier, cartographier, provoquer, affiner — est directement transposable à l'activité d'audit de code ou de test d'intrusion défensif, où l'objectif est de démontrer et de documenter une vulnérabilité pour en permettre la correction, jamais de l'exploiter contre un système réel non autorisé.

6. Prévention à la source : la meilleure défense

Au-delà des contre-mesures systémiques (qui compliquent l'exploitation mais ne l'empêchent pas toujours, comme l'illustre l'existence même du ROP), la prévention la plus fiable reste à la source, au niveau du code lui-même : fonctions bornées (`snprintf` plutôt que `sprintf/strcpy`, qui exigent explicitement de préciser la taille maximale du tampon de destination), vérification systématique des tailles avant toute copie ou allocation, compilation avec les avertissements activés et des outils d'analyse statique/dynamique (AddressSanitizer, valgrind — rappel module C, TP6, capables de détecter des débordements et des usages après libération dès la phase de développement plutôt qu'en production), et, pour de nouveaux développements sensibles, l'usage de langages à sécurité mémoire garantie par construction (Rust, Go) lorsque le contexte le permet, qui éliminent structurellement une grande partie de cette famille de vulnérabilités sans reposer sur la discipline individuelle du développeur.

À retenir

- Un débordement de tampon sur la pile peut, en l'absence de protections, permettre à un attaquant de rediriger le flot d'exécution du programme.
- Les protections systématiques modernes (canary, ASLR, DEP/NX) rendent l'exploitation bien plus difficile mais ne remplacent pas une prévention rigoureuse à la source.
- La prévention à la source (fonctions bornées, vérifications systématiques, outils d'analyse) reste la défense la plus fiable, en cohérence directe avec les pratiques déjà vues au module *Algorithmique et Programmation en C*.

Chapitre 6 — Sécurisation des API et bonnes pratiques

Sécurité des applications

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Spécificités de la sécurité des API

Les API (notamment REST et GraphQL — cf. module *Service web*) sont devenues le principal mode d'intégration entre applications, exposant souvent directement une logique métier et des données sensibles, avec une surface d'attaque distincte des applications web traditionnelles. La différence essentielle tient à l'absence d'interface graphique intermédiaire : une application web classique filtre implicitement une partie des usages possibles en ne proposant, via son interface, que certains parcours et certaines actions ; une API, elle, expose directement ses opérations à quiconque en connaît (ou en devine) le contrat, sans ce filtrage implicite. Un client mal intentionné peut appeler n'importe quel point de terminaison (*endpoint*) de l'API directement, dans n'importe quel ordre, avec n'importe quelle valeur de paramètre, sans passer par les contraintes qu'imposerait une interface utilisateur.

Cette différence a une conséquence directe sur la posture de sécurité à adopter : toute règle de sécurité qui ne serait appliquée qu'au niveau de l'interface (masquer un bouton, désactiver un champ) est totalement inefficace pour une API, puisque rien n'oblige un client à passer par cette interface. La sécurité d'une API doit donc être entièrement portée par le serveur, à chaque appel, indépendamment de la façon dont ce serveur est habituellement consommé.

Référentiel dédié, distinct du Top 10 applicatif général vu au chapitre 1, structurant les risques spécifiques aux architectures orientées API — sa nécessité vient précisément du fait que certains risques prennent une forme différente, ou une importance différente, dans une API par rapport à une application web classique :

- **Contrôle d'accès défaillant au niveau des objets (BOLA — *Broken Object Level Authorization*)** : un utilisateur accède aux données d'un autre utilisateur en modifiant simplement un identifiant dans la requête (ex. `GET /api/commandes/1234` accessible en changeant l'identifiant, sans vérification que la commande appartient bien à l'utilisateur authentifié) — variante API de l'IDOR (*Insecure Direct Object Reference*) déjà vu, et l'une des catégories les plus fréquemment rencontrées en audit d'API, précisément parce que l'authentification (vérifier qui est l'appelant) est souvent implémentée correctement alors que l'autorisation fine sur chaque objet (vérifier que cet appelant a bien le droit d'accéder à cet objet précis) est oubliée.
- **Authentification défaillante** : rappel du chapitre 3, appliqué spécifiquement aux jetons d'API (clés statiques mal protégées, jetons sans expiration, absence de révocation possible).
- **Exposition excessive de données (excessive data exposure)** : l'API renvoie plus de champs que nécessaire dans sa réponse, en s'appuyant sur le client (l'application web ou mobile) pour filtrer ce qui est effectivement affiché à l'utilisateur — un attaquant inspectant directement la réponse brute de l'API, sans passer par l'interface qui filtre l'affichage, accède alors à des données qui n'auraient jamais dû être transmises en dehors de ce qui est strictement nécessaire à la fonctionnalité concernée.
- **Absence de limitation de ressources et de débit (rate limiting)** : permet des attaques par force brute sur l'authentification, des attaques par déni de service applicatif, ou une extraction massive de données (scraping) à un rythme qu'aucun usage légitime ne justifierait.
- **Contrôle d'accès défaillant au niveau des fonctions (BFLA — *Broken Function Level Authorization*)** : un utilisateur standard parvient à appeler un point de terminaison réservé aux administrateurs faute de vérification côté serveur — une restriction uniquement appliquée côté interface utilisateur (bouton non affiché pour un utilisateur standard) n'est jamais suffisante, puisque rien n'empêche un client d'appeler directement ce point de terminaison sans passer par l'interface.
- **Mauvaise configuration de sécurité** : en-têtes de sécurité manquants, messages d'erreur trop verbeux révélant des détails d'implémentation (trace d'exécution complète, version exacte d'un composant), CORS mal configuré (autorisant des origines non nécessaires, détaillé à la section 5).

Risque BOLA/BFLA	Question de sécurité correspondante
BOLA	« Cet utilisateur a-t-il le droit d'accéder à <i>cet objet précis</i> ? »
BFLA	« Cet utilisateur a-t-il le droit d'appeler <i>cette fonction précise</i> ? »

Ces deux catégories, bien que proches, portent sur des vérifications distinctes qu'une application doit effectuer indépendamment l'une de l'autre : vérifier qu'un utilisateur peut appeler une fonction (BFLA) ne dispense pas de vérifier, en plus, qu'il a le droit d'agir sur l'objet précis visé par cet appel (BOLA).

3. Authentification et autorisation des API

Plusieurs mécanismes coexistent pour authentifier un appelant d'API, avec des compromis différents entre simplicité et granularité :

- **Clés d'API** : simples à mettre en œuvre mais offrant peu de granularité (généralement une clé donne accès à l'ensemble de l'API, sans distinction fine d'opérations) ; à transmettre exclusivement via un en-tête HTTP (jamais dans l'URL, qui peut être journalisée par des intermédiaires réseau ou mise en cache par un navigateur ou un proxy, exposant la clé bien après la requête initiale) et sur une connexion chiffrée.
- **OAuth 2.0** : cadre standard de délégation d'autorisation, permettant à une application tierce d'accéder à des ressources au nom d'un utilisateur sans jamais connaître son mot de passe — l'utilisateur autorise explicitement l'application tierce auprès du fournisseur d'identité, qui délivre ensuite un jeton d'accès limité en portée et en durée.
- **JWT** : rappel du chapitre 3, avec vérification stricte de la signature et de l'expiration à chaque appel d'API, sans exception.
- **Principe du moindre privilège appliqué aux portées (scopes)** : une clé ou un jeton d'API ne devrait donner accès qu'aux opérations et ressources strictement nécessaires à l'usage prévu, pas à l'ensemble de l'API par défaut — une intégration qui n'a besoin de lire que des données publiques ne devrait, par exemple, jamais recevoir un jeton disposant également de droits d'écriture ou d'administration.

4. Validation systématique côté serveur

Principe absolu : **toute validation effectuée uniquement côté client (JavaScript) doit être considérée comme inexistante du point de vue de la sécurité**, car un attaquant peut appeler directement l'API en contournant totalement l'interface utilisateur — par exemple avec un simple client HTTP en ligne de commande, sans jamais charger la page web ni exécuter le moindre JavaScript associé. Toute règle de sécurité (autorisation, validation de format, limites métier telles qu'un montant maximal ou une quantité en stock) doit être appliquée et vérifiée côté serveur, la validation côté client n'étant qu'un confort d'expérience utilisateur (afficher un message d'erreur immédiat, sans attendre un aller-retour réseau), jamais un contrôle de sécurité à part entière. Ce principe rejoint directement celui du chapitre 1 : la source de la donnée (le client, potentiellement contrôlé entièrement par un attaquant) ne peut jamais être considérée comme fiable par le puits qui la reçoit (le serveur).

5. Configuration CORS (Cross-Origin Resource Sharing)

Le mécanisme **CORS** définit quelles origines web sont autorisées, par le navigateur de l'utilisateur, à effectuer des requêtes cross-origin vers une API depuis une page chargée sur une autre origine. Le serveur déclare, via des en-têtes de réponse (`Access-Control-Allow-Origin` notamment), quelles origines sont autorisées à lire le résultat d'un tel appel ; le navigateur applique ensuite cette politique de son côté. Une configuration `Access-Control-Allow-Origin: *` (autorisant toute origine) combinée à l'autorisation d'envoi de cookies/identifiants (`Access-Control-Allow-Credentials: true`) est une mauvaise pratique fréquente et généralement incohérente — les spécifications interdisent d'ailleurs cette combinaison précise dans la plupart des navigateurs modernes — car elle peut faciliter des attaques exploitant le navigateur d'un utilisateur déjà authentifié depuis un site tiers malveillant, qui parviendrait alors à faire lire par sa propre page le résultat d'appels effectués avec les cookies de la victime. Une configuration CORS restrictive n'autorise explicitement que les origines réellement nécessaires à l'usage légitime de l'API, jamais un joker par défaut de facilité.

6. Limitation de débit et quotas (rate limiting)

Mesure de défense en profondeur essentielle, souvent négligée lors de la conception initiale d'une API : limiter le nombre de requêtes qu'un client (identifié par clé d'API, jeton ou adresse IP) peut effectuer dans une fenêtre de temps donnée. Cette limitation contre à la fois les attaques par force brute sur l'authentification (un attaquant ne peut plus tester un grand nombre de combinaisons en un temps réduit), les attaques par déni de service applicatif visant à épuiser les ressources du serveur, et l'extraction massive de données (scraping) qui, sans limitation, permettrait de récupérer l'intégralité d'un jeu de données via des appels légitimes mais répétés à très haute fréquence. Une limite de débit bien conçue distingue généralement plusieurs granularités (par utilisateur, par clé d'API, par adresse IP) et prévoit une réponse explicite (code de statut HTTP dédié, en-têtes indiquant le quota restant) permettant à un client légitime d'adapter son comportement.

7. Journalisation et supervision des API

Journaliser les tentatives d'accès refusées, les erreurs d'authentification répétées, et les schémas d'usage anormaux (rappel du module *Gouvernance*, chapitre 4, sur les indicateurs de sécurité), pour permettre la détection d'une exploitation en cours et alimenter une réponse à incident. Une journalisation utile pour la sécurité enregistre systématiquement l'identité de l'appelant (ou son adresse d'origine si non authentifié), le point de terminaison appelé, le résultat de la vérification d'autorisation, et l'horodatage — sans toutefois enregistrer en clair des données sensibles (mots de passe, jetons complets, données personnelles), qui ne devraient jamais apparaître directement dans un journal.

8. Synthèse : une checklist de sécurisation d'API

1. Authentification forte et jetons correctement vérifiés (chapitre 3).
2. Autorisation vérifiée systématiquement à chaque appel, au niveau objet et fonction (jamais déléguée au client) — cf. BOLA et BFLA (section 2).
3. Validation stricte des entrées côté serveur, quelle que soit la validation déjà présente côté client (section 4).
4. Exposition minimale des données (ne renvoyer que les champs nécessaires à l'usage prévu).
5. Limitation de débit et quotas (section 6).
6. Configuration CORS restrictive et justifiée, sans combinaison dangereuse origine-joker et identifiants (section 5).
7. Chiffrement systématique des communications (TLS, module *Cryptographie*).
8. Journalisation et supervision des accès et anomalies (section 7).

Cette checklist n'est pas une simple liste de contrôles indépendants à cocher isolément : elle reprend, appliquée spécifiquement au contexte des API, l'ensemble des principes vus dans ce module — validation systématique des entrées non fiables (chapitres 1 et 2), authentification et gestion rigoureuse des sessions et jetons (chapitre 3), et défense en profondeur combinant plusieurs couches de contrôle indépendantes (chapitre 1).

À retenir

- L'OWASP API Security Top 10 recense des risques spécifiques aux API (BOLA, BFLA, exposition excessive de données, absence de rate limiting), complémentaires du Top 10 applicatif général.
- Aucune validation ni autorisation effectuée uniquement côté client ne doit être considérée comme un contrôle de sécurité : tout doit être revérifié côté serveur.
- La sécurisation d'une API repose sur la combinaison systématique de plusieurs contrôles (authentification, autorisation, validation, limitation de débit, configuration, journalisation), en cohérence directe avec la défense en profondeur vue au module *Security by design*.